



Module 3 Day 3: Commanding the Command Line

Overview

Today, you will expand your command-line skills by working in the terminal to complete tasks that require file searching and bulk operations. You will complete a series of activities involving the `man`, `find`, `grep`, and combining multiple commands in sequence.

Class Objectives

By the end of class, you will be able to do the following:

- Identify and explain the structure of a terminal command
- Explain how options modify the default behavior of a terminal command
- Use the `man` command to list instructions and options for each command
- Use the `find` command to locate files based on search parameters
- Use the `grep` command to search within the contents of files
- Combine multiple commands in sequence with pipes to accomplish intermediate IT tasks

01. Warm-up (Lab)

In this activity, you will continue with the scenario in which you are a security analyst at Candy Corp. Candy Corp's management apprehended Ruth and Henry, and is grateful for all your work so far. The local authorities have video evidence that Sugar delivered a cash payment to Candy Corp's back door on October 13, 2019.

You are tasked with gathering physical access logs to prove Henry or Ruth opened the back door for the delivery. This will assist the authorities with solidifying their case.

- [Activity File: Warm-up](#)

02. The `find` Command

We have been navigating in and out of multiple directories in order to find specific files and directories, but this process is very time-consuming and some file systems have hundreds of directories to search through.

- For example, you may be asked to find access logs in a server you are not familiar with. Often, security professionals are not provided the exact location of a file, so you may have to navigate through hundreds of directories to find the access logs.

There is a terminal command, called `find`, that is designed to simplify this task by searching for specific files and directories with a single command.

- By default, `find` will search through the current directory and its subdirectories.
- However, `find` does not look at the contents of a file; it only uses the file name or directory name.

In the following examples, we will use the same base command, `find`, but we will slightly modify the command to achieve different results.

Syntax for finding a file

1. `find -type f`
 - We will start with the command used to find **all files**.
 - In this example, we are finding all files in our current directory and its subdirectories.
 - In order to do this, we will use the option, `-type`, and the required parameter, `f`, to indicate that we are searching for files.
2. `find -type f -name log.txt`
 - Now, we will find a **specific file**.
 - In this example, we are finding the files called `log.txt` in our current directory and its subdirectories.
 - In order to do this, we are using the option, `-name`, to search for an exact match of the specified parameter, `log.txt`.
3. `find -type f -iname log.txt`
 - To remove case sensitivity and find all relevant files, regardless of whether the case of a file name matches the parameter's case, we will change the `-name` option to `-iname`.
 - This option requires a parameter identifying what we are looking for.
 - We can use the above example to find the files called `log.txt` (lowercase) and `LOG.TXT` (uppercase) in our current directory and its subdirectories.
4. `find -type f -iname '*.txt'`
 - In this example, we will use a symbol known as a **wildcard** to search for all files that end with `.txt`.
 - The `*` wildcard symbol indicates that any file ending with `.txt` will be included in the results, regardless of what comes before `.txt`.

- Using wildcards with `find` is known as a wildcard search.
 - We can run the above example to find all files ending with lowercase `.txt` or uppercase `.TXT` in our current directory and its subdirectories.
 - It is important to mention that running the above command without the single quotes `'` may appear to work correctly, but it could potentially provide unintended results.
 - This can be an issue when running any command using wildcards because of how the Bash Shell handles file globbing.
 - It is good practice to get into the habit of using single quotes `'` whenever using a wildcard (e.g., `?` or `*`).
 - At times, you may need to search for part of a file name.
 - For example, you may want to look for all file names that begin with a certain date, regardless of what the file name ends with.
 - This can be done with a wildcard, signified by an asterisk `*`.
 - Wildcards can come before text, such as `*.txt`, or after text, such as `0517*`.
 - As with the above example, if we used the `find` command to search for `*.txt`, the command might return the following:
 - `log1.txt`
 - `apachefile.txt`
 - `FILEAB.txt`
 - If we used the `find` command to search for `0517*`, the command might return:
 - `0517apache.log`
 - `0517textdata.txt`
5. `sudo find /root/desktop -type f -iname log.txt`
- In this final example, we will use `find` to search for a file located in another directory. Specifically, we will find the case-insensitive `log.txt` in the `/root/desktop` directory.
 - To do this, we place the desired directory after the `find` command and before the `-type` option.

Syntax for finding a directory

1. `find -type d`
 - The syntax for finding a directory is exactly the same as the syntax for finding a file, except that the option, `-type`, requires the parameter, `d`.
 - The above command will find all directories in our current directory and its subdirectories.
2. `find -type d -name logs`
 - To find a specific directory name, we add the option `-name`, which requires the parameter of the name of the directory.
 - With the `-name` option, `find` will search for an exact, case-sensitive match of the parameter we specify.
 - In this example, we will search for a directory called `logs` in our current directory and its subdirectories.

3. `find -type d -iname logs`
 - To find a specific directory with **case insensitivity**, you will change the `-name` option to `-iname`, which also requires the parameter of the directory we are searching for.
 - In this example, we will look for a directory called `logs` or `LOGS`.
4. `find -type d -iname '*1013'`
 - The `*` symbol indicates this is a wildcard search.
 - In this example, we will search for all directories that end with the date `1013`.
 - The command will output directories ending in `1013`, despite what comes before, including directories such as the following:
 - `apache1013`
 - `textdata1013`
5. `sudo find /root/desktop -type d -iname logs`
 - Lastly, we will search for a directory within a directory that we are not currently located in.
 - Specifically, this command finds directories called `logs` in the `/root/desktop` directory.

Summary:

- `find` is used to locate a file or directory.
 - `type f` is the option used to find files
 - `type d` is the option used to find directories
 - `name` is an additional option used to find specific file or directory names
 - `iname` is an additional option used to find case insensitive names
-

03. Activity: find Your Way (Lab)

In this activity, you will continue to play the role of a security analyst at Candy Corp. Your manager has identified why the website, PeanutButtery.net, was targeted: Henry had hidden a secret recipe for a revolutionary candy within the file system, and Sugar was trying to find it.

Your manager at Candy needs you to search through the files and file directories of the PeanutButtery.net server to find the hidden recipes in its file system. You must scan the directories of the site PeanutButtery.net to uncover the secret recipes hidden by Henry.

- [Activity File: find Your Way](#)
-

04. Review: find Your Way

The instructor will review the activity with the class.

05. The `grep` Command

The `find` command only searches for the names of files, not their contents. However, IT and security professionals are often tasked with searching for specific data inside a file.

- For example, you might be tasked with finding out whether a specific user logged in on a specific day. You would first need find the access log file for that day. Then, you would need to verify whether that specific user was inside the log file.

We have previously used preview commands, such as `head`, `more`, `tail`, and `less`, to view the contents of a file. The challenges of using preview commands to search for data inside a file include the following:

- Large files take a long time to scan for data.
- If you have more than one file to scan, it can take a long time to preview multiple files.
- Manually previewing and scanning files invites human error. Even the best security professionals can overlook a data point if it blends in with the other data in the file.

There is a command, called `grep`, that allows us to search within a file, or multiple files, to find a specific data point.

- `grep`, which stands for "global regular expression print," is a command used to search for data inside files.
- `grep`, by default, returns the entire line that the desired data point is found in.
- `grep`, by default, will only search for data in the current directory, and not in its subdirectories.

We will now walk through the syntax for four methods we can use to find a data point with `grep`.

Syntax for `grep`

```
grep <data_point> <File_to_search_inside>
```

- In this syntax, `grep` is used to find a specific data point within a **single** file.
- Next, we will run through a few examples of the `grep` command, highlighting the various features and options we can use with it.

1. `grep bob log1.txt`
 - In this example, we are using `grep` to find the lines in which the user `bob` is mentioned in the file `log1.txt`.
 - We know the following from the syntax:
 - `grep` is the command being run.
 - `bob` is the specific data point being searched for.
 - `log1.txt` is the file being searched for the data point.

- After this command is run, all the lines where the data point, `bob`, was found inside the file, `log1.txt`, will be displayed.
 - If no matches of `bob` are found in the file, nothing will be returned.
2. `grep bob *.txt`
- In this example, we are using `grep` to find a specific data point within **multiple** files.
 - Specifically, we are using `grep` to find where `bob` exists within all `.txt` files.
 - We know the following from the syntax:
 - `bob` is the specific data point being searched for.
 - `*.txt` is the wildcard. `*` indicates that the command will search through all files that end with `.txt`.
 - After this command is run, it will display the files where the value of `bob` was found, followed by the lines where it was found inside all the `.txt` files.
3. `grep -i bob *.txt`
- In this example, we are using `grep` to search all `.txt` files to find a case-insensitive data point.
 - We know the following from the syntax:
 - `grep` is the command being run.
 - `-i` is the option for `grep` that indicates case insensitivity.
 - `bob` is the specific data point being searched for.
 - `*.txt` is the wildcard that indicates this command will search through all files that end with `.txt`.
4. `grep -il bob *.txt`
- In this final example, we are using `grep` to find the **file name** that contains a specific data point.
 - Specifically, this command only outputs the **file name** where the user of `bob` or `BOB` exists within all `.txt` files. When this command is run, it will only display the names of the files that contain `bob`.
 - We know the following from the syntax:
 - `grep` is the command being run.
 - `-il` represents two options that are placed together:
 - `i` is the option for `grep` that indicates case insensitivity.
 - `l` is the additional option that indicates to only return the file name
 - Note that although `i` and `l` are two separate options, we can place them under a single hyphen.
 - `bob` is the specific data point being searched for.
 - `*.txt` is the wildcard that indicates this command will search through all files that end with `.txt`.

Summary:

- `grep` is the command-line command used to find a data point inside a file.
 - The basic syntax is as follows:


```
<grep data_point File_to_search_inside>
```

- `grep`, by default, will return the **whole line** containing a data point.
 - `i` is the option indicating that `grep` must conduct a **case-insensitive** search for the data point.
 - `l` is the option indicating that `grep` must return only the file names of the files containing the data point.
-

06. Activity: `grep` (Lab)

In this activity, you will continue in the role of a security analyst at Candy Corp. Your manager found out that Sugar Corp recently made a large purchase of guavaberries. They currently do not have any candy that includes guavaberries, but Candy does. Your manager suspects that Sugar is trying to reproduce some of Candy's secret recipes that use guavaberries.

Your task is to determine which of the secret recipes contain guavaberries in their list of ingredients.

- [Activity File: `grep`](#)
-

07. Review: `grep`

The instructor will review the activity with the class.

08. Combine Commands with Piping

We have covered some of the powerful command-line commands that IT and security professionals use, including the following:

- `find`: This command searches for file names or directories.
- `grep`: This command searches for data points inside files
- `wc`: This command counts lines or words inside a file.

Security professionals are often tasked with combining these commands to complete certain tasks.

- For example, you may be asked to determine whether a user exists in a log file and how many times that user appears in the log file.
 - We can use the `grep` command to see whether a user appears in the log file by redirecting the results into an output file.
 - We can use the `wc -l` command to determine how many times the user appears in the file by counting the results of this output.

Rather than running these multiple commands separately, we can combine them using **pipes**.

- A pipe takes the output of one command and redirects it to another command in order to complete additional tasks on the output.
 - A pipe is designated with the following symbol: |
 - Multiple pipes can be used in a single command.
 - Pipes are unidirectional, meaning the processing of the data only flows from left to right through the pipeline.
-

09. Activity: Gather Evidence (Lab)

In this activity, you will continue in your role as a security analyst at Candy Corp. Candy Corp believes it has enough evidence to send to the authorities to charge Sugar with a cyber crime.

Your task is to gather several points of evidence from your file systems, which Candy Corp will provide to the authorities to prove that Sugar is stealing data.

- [Activity File: Gather Evidence](#)
-

10. Review: Gather Evidence

The instructor will review the activity with the class.

11. Looking Forward

The commands you learned today are critical to conducting many tasks that IT and security professionals do on a daily basis, and the best way to get comfortable with them is to practice. It is necessary to continue practicing outside of class to build your proficiency.

In the next module, you will learn about permissions, file ownership, and access controls.